Milestone 1 spec — task-start flow

Oct 14, 2026 · @Jesus Gonzalez

Goal and context

When you describe a task in plan mode, Claude Code runs the full task-start flow before it shows you a plan: recall, library docs and gotchas, pointed questions if needed, then a council audit. This works whether the repo uses a local AgentOps or your server.

This is milestone 1 of the cue-driven memory requirement (written 2026-10-14). That doc holds the problem, the evidence and the verification findings; this spec only covers what to build.

Decisions already made (do not revisit):

- Plan mode always runs the full flow, with no size check.
- Enforcement is soft. Claude Code cannot reliably block `ExitPlanMode`, so the flow is injected and checked, never hard-gated.
- The server is the primary target. Hooks must work on repos connected with `connect --remote`.
- Hooks recall through graph edges and lexical search only. No embedding model inside a hook call.
- Council members get the full read-only agentops toolset.
- Pointed questions have a trigger rule and a cap.

Based on commit `ca098e9` (2026-10-13).

Work items

Seven items, in this order. Item 0 is a spike whose findings can change items 3 and 6; nothing from it is merged.

#	Item	Main files
0	Spike: confirm four Claude Code hook behaviours on the installed version	none (findings note only)
1	Route hooks to the server when the repo is remote-connected	`agentops-cli/src/hook_capture.rs`, `agentops-cli/src/main.rs` (marker helpers), `agentops-heavy-api` (new routes), `agentops-api` (parity)

#	Item	Main files
2	Plan-mode trigger: inject the flow and a recall seed on `UserPromptSubmit`	`hook_capture.rs`, `agentops-retrieval` (lexical recall), `agentops-mcp/src/session_guide.rs`
3	Council and subagents: read toolset in frontmatter, seed context on `SubagentStart`	`prompts/agents/plan-council-member.md`, `plan-council-chair.md`, `hook_capture.rs`, `maybe_install_claude_hooks`
4	Library gotchas: filterable `list_gotchas`, `library-docs` loads them	`agentops-mcp/src/tools.rs`, `prompts/shared/library-docs.md`
5	Pointed questions inside the flow, with a cap	`prompts/shared/plan.md`, `prompts/shared/interview-me.md`
6	Soft enforcement: check the presented plan, nudge once, then warn	`hook_capture.rs`, `maybe_install_claude_hooks`, `prompts/shared/plan.md`, `audit-plan.md`

Each item updates the matching engineering doc in the same branch: `13-agent-integration.md` (1, 2, 3, 6), `07-mcp-and-rest-contracts.md` (1, 4), `06-notes-and-consolidation.md` (4).

Behavior

0. Spike

On the Claude Code version you use, with a throwaway hook that logs its stdin, confirm:

1. `UserPromptSubmit` input carries `permission_mode`, and it reads `"plan"` in plan mode.
2. `SubagentStart` fires for Explore and for `plan-council-member`, what it matches on, and that its `additionalContext` reaches the subagent.
3. Which hook sees the plan when it is presented: `PermissionRequest` matched on `ExitPlanMode` (reading `tool_input`), `Stop` (reading `last_assistant_message`), or neither.
4. Whether `Stop` input carries `stop_hook_active`.

Write the answers to `docs/engineering/spikes/m1-hooks.md`. If 1 fails, stop and report: item 2 has no trigger. If 2 or 3 fail, drop that part of item 3 or 6 and say so in the plan.

1. Hook routing

- `hook-capture` reads `.context/agentops-remote.json` first, with the same marker type and path as `connect`. Move the marker helpers out of `main.rs` into a shared module; don't duplicate them.
- Marker present: every read and write goes to the server over HTTP with the marker's API key. The local `.context/graph.db` is never opened.
- Marker absent: today's local behavior, unchanged.
- A network error, timeout or non-2xx response never fails the hook. It logs one line to stderr and exits 0 with no context. Recall is best-effort; the session must never stall on it.
- This covers every existing event (`posttooluse`, `posttoolusefailure`, `userpromptsubmit`, `sessionstart`, `pretooluse`), not only the new ones.

2. Plan-mode trigger

On `UserPromptSubmit` with `permission_mode == "plan"`:

- **First prompt of a planning episode** (the previous recorded prompt in this session was not in plan mode, or there is none): inject the flow directive and a recall seed.
- **Later prompts in the same episode** (answers to questions, refinements): inject only a one-line reminder of the current step. Never restart the flow.
- Record each prompt's mode as part of its `user_intent` session event, so the next prompt can tell whether an episode began.

The **flow directive** is fixed text, kept in `prompts/shared/plan-flow.md` and compiled into the binary. It tells the agent, in order: use the `plan` skill; recall first; run `library-docs` for every library the task touches; ask pointed questions if the trigger rule fires; run `audit-plan`; present the plan with its "Research performed" and "Council verdict" sections.

The **recall seed** is pointers, not content: up to `PLAN_SEED_MAX_ITEMS` gotchas, decisions, notes and doc sections that match the prompt. Lexical (BM25 plus exact-name) over those node kinds, plus exact-name matches on symbols the prompt names, followed one hop to the notes that affect them. Each pointer is one line: kind, title, node id, and why it matched. Pinned notes stay in front, as today.

3. Council and subagents

- `plan-council-member` and `plan-council-chair` list every read-only agentops tool by full name (`mcp__agentops__<tool>`), next to their existing tools. The list is the read-only tools from `07-mcp-and-rest-contracts.md`, core and docbrain. No write tool.

- Their prompts tell them to look up nodes for each file they review (`get_symbol` , `related_context`) before using `Grep` .
- Fix the prompts' claims of "scoped `list_gotchas` ". That tool takes no scope today; item 4 adds one, and the prompts use it.
- `SubagentStart` (if the spike confirms it): for Explore and council agents, inject the session's pinned notes and the recall seed from item 2, as pointers, inside `SUBAGENT_SEED_BUDGET` .

4. Library gotchas

- `list_gotchas` gains two optional inputs: `query` (lexical match on title and body) and `tag` (exact tag match). With neither, output is unchanged.
- `library-docs` adds a step after docs are found: `list_gotchas` with `tag` set to the library slug, then with `query` set to the library name. Report what came back, or one line saying nothing did.

5. Pointed questions

- Trigger: the task is missing its outcome, its scope boundary, or a constraint the plan depends on, and recall did not answer it. Use `interview-me` 's existing criteria.
- Ask one question at a time with `AskUserQuestion` , with a guess attached, up to `MAX_PLAN_QUESTIONS` . Past the cap, plan anyway and list the open assumptions at the top of the plan.
- **Conflict to resolve:** `interview-me` ends with a statement of intent, waits for an explicit yes, and "stops the turn" without starting downstream work. Inside the plan flow it must skip that gate and go on to planning. Add a plan-flow mode to `interview-me` rather than a second skill.

6. Soft enforcement

- The plan template gains a required "Council verdict" section next to "Research performed". `audit-plan` writes its verdict there.
- Use whichever hook the spike showed can see the presented plan. If either section is missing, the hook nudges once: it asks the agent to finish the missing step. If the agent presents again without it, the hook lets it through and shows you a warning (`systemMessage`) naming what was skipped.
- Never more than `ENFORCE_MAX_NUDGES` nudge per planning episode, so this can't loop.
- `connect --with-hooks` installs the new events (`SubagentStart` , and `PermissionRequest` or `Stop` per the spike) and stays idempotent.

Contracts and parameters

New server routes (item 1)

These follow the existing `/repos/{id}/usage/sync` pattern: `{id}` is the marker's `connection_id`, and auth is `Authorization: Bearer <api_key>`. Each route calls the same use-case function the local path calls, so the two paths can't drift.

Route	Request	Response	Used by
`POST /repos/{id}/session-events`	`{session_id, kind, description, category, permission_mode?}`	`204`	Every capturing hook
`GET /repos/{id}/session-context`	`session_id`, `parts` (`pinned`, `guide`, `last_mode`)	`{pinned, guide, last_mode}`, each omitted if empty	`sessionstart`, `userpromptsubmit`, `subagentstart`
`GET /repos/{id}/recall`	`q`, `limit`	`[{node_id, kind, title, reason}]`	Recall seed (items 2, 3)

If you find that hooks calling the existing `/mcp` endpoint fits better than new routes, argue it in the plan; don't switch silently. Add the routes to `docs/openapi.yaml` and regenerate `docs/index.html`.

Hook output

Every hook output that injects context uses `hookSpecificOutput.additionalContext`, the same shape as today's `sessionstart` and `userpromptsubmit` handlers. Every hook exits 0 except the item 6 nudge, which uses whichever block mechanism the spike confirmed.

`list_gotchas` (item 4)

`path*`, `session_id`, `query`, `tag`. `query` and `tag` combine with AND. Read-only and Advisor, as today.

Parameters

These are starting values. Each one is a named constant in code; change them in one place.

Name	Value	Why
`PLAN_FLOW_BUDGET`	4,000 characters	Directive plus seed plus pins; well under Claude Code's 10,000 cap on prompt context
`PLAN_SEED_MAX_ITEMS`	8	Pointers the agent can scan at a glance
`SUBAGENT_SEED_BUDGET`	2,000 characters	Same as the session guide budget
`MAX_PLAN_QUESTIONS`	3	Enough to pin scope without stalling the plan
`ENFORCE_MAX_NUDGES`	1 per planning episode	A nudge, not a loop
`REMOTE_HOOK_TIMEOUT`	3 s per HTTP call	A prompt hook must never feel slow
Hook `timeout` in settings	10 s for `UserPromptSubmit` and `SubagentStart`	Below Claude Code's 30 s default; a failure still lets the prompt through
Recall latency target	p95 under 300 ms local, under 1 s remote	Measured in the done-when checks

Done when

Automated (`cargo test --workspace` passes, plus these new tests):

- [ ] `hook_capture` unit tests, fed fixture payloads on stdin:
  - plan-mode first prompt emits the directive plus seed within `PLAN_FLOW_BUDGET`;
  - a second plan-mode prompt emits only the reminder;
  - a default-mode prompt emits only pinned notes, exactly as today.
- [ ] With a marker file pointing at an unreachable URL, every event exits 0 within `REMOTE_HOOK_TIMEOUT` plus 1 s, prints one stderr line, and never creates `.context/graph.db`.

- [ ] Integration test: an `agentops-server` started in-process with Postgres, a repo connected through a marker, and the user-prompt hook. Its session event lands in the server's store, and the seed comes from the server's graph.
- [ ] `list_gotchas` with `tag`, with `query`, with both, and with neither (the last byte-identical to today's output).
- [ ] A `ruler-bridge` test asserting the rendered `plan-council-member` and `plan-council-chair` list every read-only agentops tool and no write tool.
- [ ] `connect --with-hooks` run twice leaves one entry per event in `.claude/settings.json`.

Measured:

- [ ] The recall seed meets the latency target over 50 prompts against this repo's own graph, local and remote. Report p50 and p95.

Manual, on your machine against your server:

- [ ] In plan mode, a one-line task ("add a `--json` flag to `agentops status`") produces, before any plan: a recall seed, a `library-docs` check, an audit, and a plan with both sections.
- [ ] A vague task ("make search better") triggers questions, at most 3, then a plan that lists its open assumptions.
- [ ] Council members' transcripts show agentops tool calls before `Grep`.
- [ ] A plan presented without "Council verdict" gets exactly one nudge, then a visible warning.
- [ ] Replay the evidence sessions from the requirement: the missed memory appears in the seed or in a council member's context before the step that went wrong.

Docs: the three engineering docs named under Work items are updated, and `openapi.yaml` includes the new routes.

Out of scope

- Cued recall during work on file reads, edits and errors. That's milestone 3; item 2's lexical recall function is the piece it will reuse.
- MEMORY.md import (milestone 2), reinforcement on use, and automatic consolidation at session end (milestone 4).
- Any embedding model call inside a hook.
- Agents other than Claude Code.
- A hard block on `ExitPlanMode`.
- Changing the council's size gate or personas.

Instructions to the agent

1. Run item 0 first and show me the findings before writing any other code.
2. Then propose an implementation plan covering items 1 to 6. Use this repo's own `plan` and `audit-plan` skills on it, and include its "Research performed" section.
3. In the plan, flag anything in this spec that contradicts the code, and anything you think is wrong.
4. Wait for my approval. Then implement one item per commit, in order, each with its tests and doc update.

Handoff report for milestone 2

When all six items are merged, write `docs/engineering/handoffs/m1-report.md` and stop. It feeds the milestone 2 spec (MEMORY.md import), which is drafted outside this repo, so it has to stand on its own. Facts only, each with a file path, commit or command output. Mark anything unverified as unverified.

1. What shipped

- One line per item with its commit hash.
- Every deviation from this spec: what changed, why, and who approved it.
- The spike findings, linked to `spikes/m1-hooks.md`, plus anything learned about hooks after the spike.

2. Interfaces milestone 2 will reuse. Exact signatures, not descriptions:

- The lexical recall function: crate, name, inputs, output, and which node kinds it searches.
- The hook routing layer: how a caller picks local or remote, and how to add a new remote call.
- The three server routes as built, and any others added.
- `list_gotchas`'s final inputs, and how tags are stored on a note.

3. Measurements

- Recall seed latency, p50 and p95, local and remote, and the command that produced them.
- Size of this repo's graph at measurement time (node count by kind).
- Outcome of every manual check under Done when: pass or fail, with a sentence on what happened.
- The evidence-session replay: for each session, whether the missed memory appeared, and where.

4. Current state of memory writing. Milestone 2 imports existing memories, so it needs to know how a memory gets written today:

- Every path that creates a gotcha, decision or note, local and remote (`add_note`, `ingest_notes`, `note` CLI, vault files). Inputs, auto-classification behavior, and whether it de-duplicates.
- How symbol matching works for a new note, and what happens to a note that matches no symbol.
- What curation states exist (`Pinned`, `Reduced`, others), and whether there is anything usable as a "needs review" queue.
- How `AGENTS.md` is generated and distributed through Ruler, and whether regenerating it overwrites lines added by hand. Procedural memories will be written there.

5. The MEMORY.md files to import. Read-only inventory. Do not move or change anything:

- Every MEMORY.md Claude Code keeps on this machine (user level and per project), with path, size, and entry count.
- Five representative entries per file, quoted, with secrets redacted.
- Your rough split of entries into facts, past events and rules, with the count of each. Label it a guess.

6. Open issues

- New findings, bugs and debt, also added to `findings.md`.
- Anything in milestone 1 that milestone 2 must not break.
- Questions you would want answered before the milestone 2 spec is written.